CORS in ASP.Net Core Web API

Overview



What is CORS Policy?

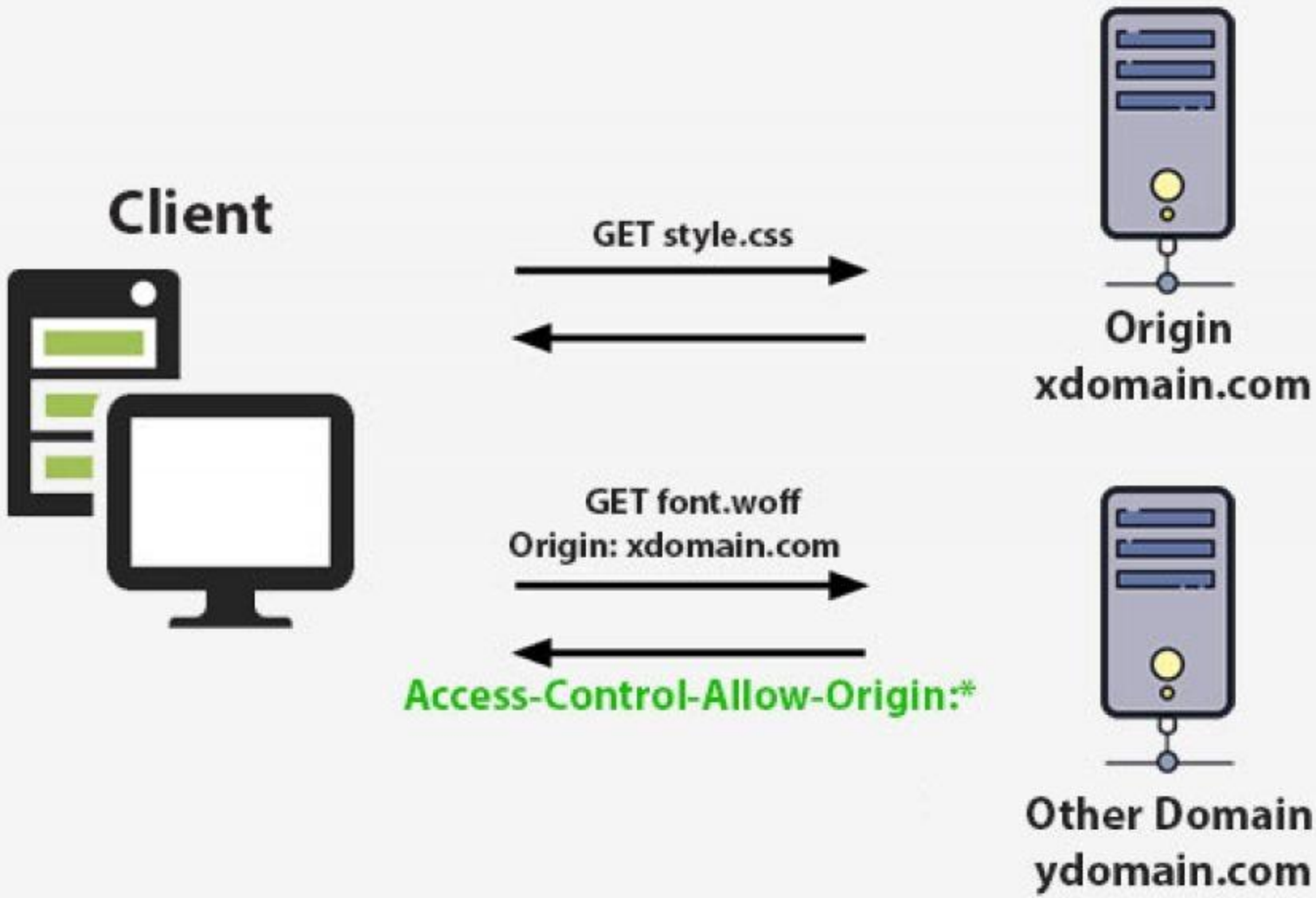
Why we need to apply CORS Policy?

How to apply CORS Policy?

What is CORS Policy?

- **Cross-Origin Resource Sharing (CORS)** is a mechanism that allows restricted resources on a web page to be requested from another domain outside the domain from which the resource originated.
- In ASP.NET Core Web API, you can configure CORS policies to control which origins, methods, headers, and other request attributes are allowed to access your API.
- CORS policy is necessary to ensure security and control access to resources (e.g., APIs) from web applications hosted on different domains.

```
Access to XMLHttpRequest at 'http://localhost:5000/global_config'   step1:1
from origin 'http://localhost:8080' has been blocked by CORS policy:
Response to preflight request doesn't pass access control check: No 'Access-
Control-Allow-Origin' header is present on the requested resource.
```


Client
GET style.css
Origin
xdomain.com
GET font.woff
Origin: xdomain.com
Access-Control-Allow-Origin:*
Other Domain
ydomain.com

Why we need to apply CORS Policy?

- However, there are valid scenarios where you might want to allow cross-origin requests, such as when building a client-side application (e.g., JavaScript or Angular) that needs to consume data from an ASP.NET Core Web API hosted on a different domain.
- In such cases, you need to apply CORS policy to explicitly allow cross-origin requests.
- Here are some key reasons for applying CORS policy:

- **Security:** CORS policy helps protect your API by enforcing restrictions on which domains can access it. Without CORS policy, any website or web application could potentially make cross-origin requests to your API, leading to security vulnerabilities like cross-site request forgery (CSRF) attacks. Applying CORS policy allows you to define and control the trusted domains that are allowed to access your API.

- **Cross-Domain Communication:** CORS policy enables web applications hosted on different domains to communicate and share resources. It allows client-side code to access APIs and retrieve data securely from different origins. This is especially useful in scenarios where you have separate front-end and back-end applications, and they need to exchange data.

- **Access Control:** CORS policy gives you fine-grained control over which HTTP methods, headers, and origins are allowed to access your API. You can specify which domains (origins) are permitted, which methods (e.g., GET, POST) are allowed, and which headers (e.g., Authorization) can be included in the requests. This control helps prevent unauthorized access and protects sensitive data.

- **Compliance with Browser Security Mechanisms:** Applying CORS policy ensures that your API adheres to browser security mechanisms. Web browsers enforce the same-origin policy, and if a cross-origin request is made without proper CORS policy, the browser blocks the request. By configuring CORS policy correctly, you can allow browsers to make cross-origin requests to your API without violating the same-origin policy.

How to Apply CORS Policy?

Startup.cs

```
public void ConfigureServices(IServiceCollection services)
{

        //default CORS Policy
        //services.AddCors(options => {
        //options.AddDefaultPolicy(builder =>
        //{
        //builder.AllowAnyOrigin().AllowAnyHeader().AllowAnyMethod();//all front-end apps can consume web api
        //builder.WithOrigins("http://localhost:60685", "http://localhost:60685").WithMethods("GET", "POST", "PUT",
        "DELETE").AllowAnyHeader();

        //});
        //});

        //named CORS policy
        services.AddCors(options =>
        {
        options.AddPolicy(name:"AllowAny" ,builder =>
        {
        builder.AllowAnyOrigin().AllowAnyHeader().AllowAnyMethod();//all front-end apps can consume web api

        //builder.WithOrigins("http://localhost:60685", "http://localhost:60685").WithMethods("GET", "POST", "PUT",
        "DELETE").AllowAnyHeader();

        });
        });
}
```

- **To configure Pipeline.**

```
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
                        if (env.IsDevelopment())
                        {
                        app.UseDeveloperExceptionPage();
                        }

                        app.UseRouting();

                        //default cors policy
                        //app.UseCors();

                        //named cors policy
                        app.UseCors("AllowAny");

                        app.UseEndpoints(endpoints =>
                        {
                        endpoints.MapDefaultControllerRoute();
                        });
}
```

- **To apply CORS policy.**

EmployeeController.cs

```
[Route("api/[controller]")]
[ApiController]
[EnableCors("AllowAny")]
        public class EmployeeController : ControllerBase
        {
                //your logic goes here
        }
```

References

https://learn.microsoft.com/en-us/aspnet/core/security/cors

https://www.c-sharpcorner.com/article/enable-cors-consume-web-api-by-mvc-in-net-core-4/

https://code-maze.com/enabling-cors-in-asp-net-core/

https://geeksarray.com/blog/how-to-setup-cors-policies-in-aspnet-core-web-api

https://www.yogihosting.com/aspnet-core-enable-cors/

https://medium.com/@alistairrobson_36819/how-to-add-cors-to-a-dot-net-core-web-api-15312734c796

https://www.infoworld.com/article/3327562/how-to-enable-cors-in-aspnet-core.html